SENTINEL Lab 1 Brief: `libsyscall`

ECE 476 Engineering System Software • Fall 2026 • University of Victoria

Lab session	Friday, October 9, 2026
Prelab due	Wednesday, October 7, 2026, by midnight, on Brightspace. Required, with no marks[1]
Deliverable due	Thursday, October 22, 2026, by midnight, on Brightspace: 4% of the grade, out of 80 marks
Oral exam	In the Lab 2 session, Oct 30: 2% of the grade, out of 20 marks[2]
Lab 1 total	6% of the course grade: the deliverable and the oral exam

This brief states the rules of Lab 1 that no repository file states. Read it with `src/libsyscall/include/sentinel/syscall.h`, which states what each function does and each of its errors. `README.md` lists the other documents of the course.

1 The task

Lab 1 builds `libsyscall`, the layer that reads the feed (a file of contact reports), gives the time, and writes the mission log (requirement R6).[3]Write these files in `src/libsyscall/`:

- `src/io.c`, `src/clock.c`, `src/log.c` and `src/proc.c`: the 15 functions that `syscall.h` declares, each in the file that `syscall.h` names.
- `Makefile` (Section 4.1), and four tests in `tests/` (Section 4.2).
- `tests/demo_lab1.c`, the demonstration program (Section 3).

Do not change a declaration in `syscall.h`: the tests that mark your lab call each function as it declares it. You can call each function of `common/`, for example `contact_parse` (`common/contact.h`) and `lr_next_line` (`common/linereader.h`). Read `common/linereader.c` before you write the demo.

2 The rules that syscall.h does not state

- Call `read` and `write` directly in `sc_read_full` and `sc_write_all`, not `fread`, `fwrite` or a different library function. The tests that mark this lab give their own `read` and `write`, which move a few bytes at a time and fail with `EINTR`.
- `sc_now_wall` and `sc_stamp` read `CLOCK_REALTIME`. `sc_now_ns` reads `CLOCK_MONOTONIC`.
- Each function that `syscall.h` does not declare is `static`.
- The library does not print, and it does not exit: no `printf`, `fprintf`, `perror` or `exit` in `src/`. A function reports an error with its return value and `errno`.
- Examine the result of each system call. Each call that can fail has a branch for the failure. A call that cannot fail needs no branch, but a comment says why. An example is `clock_gettime()` with `CLOCK_MONOTONIC` in `sc_now_ns`, which `syscall.h` says cannot fail.
- When a function fails after it started its work, it closes each descriptor that it opened, and it frees the memory that it allocated. Save `errno` before the cleanup, and put it back after.
- A comment says why, not what.

3 demo_lab1

```
build/demo_lab1 <feed-file> [mission-log] [--follow IDLE_MS]
```

This section is the full specification of the demo. The acceptance test (Section 4.3) compares the output of your demo with these rules, character for character.

[1]The Lab 1 prelab, `docs/lab1/prelab1.pdf` in your repository. It is due before the Lab 1 session.

[2]The oral exam of a lab is in the session of the next lab: for Lab 1, the Lab 2 session. The Submission Guide, Section 3, `docs/submit/submit.pdf`, describes it.

[3]R6 is a requirement of SENTINEL: each action has a timestamp, and the centre writes it to the mission log. The SENTINEL Lab Manual, Section 1.2, `docs/manual/sentinel_manual.pdf`, lists the six requirements.

3.1 The arguments

<feed-file> is required. [mission-log] is optional, and its default is /tmp/sentinel_mission.log. --follow IDLE_MS is optional, before, between or after the other two. IDLE_MS is a whole number from 1 to 2147483647. Each other argument is a file name: the first is the feed, and the second is the mission log. --follow=3000 is a file name. If --follow occurs two times, the second value counts, but each --follow needs a correct value.

In each of these cases, the demo writes a usage line to standard error, and it exits with status 2. It does not open the mission log.

- There is no feed argument, or there is a third file name.
- A --follow has no value, or its value is not a whole number from 1 to 2147483647.

You choose the text of each message on standard error.

3.2 The steps

Do these steps in this order. "Fail" gives the action when the step fails.

1. Open the mission log with log_open. *Fail:* write the reason to standard error, and exit with status 1.
2. Write the start line to the mission log.
3. Call proc_report(0, &before), and write the process line from before. This is the *before* count of the leak check. *Fail:* write the reason to standard error, close the mission log, and exit with status 1.
4. Open the feed with sc_open_read. *Fail:* write the reason to standard error, close the mission log, and exit with status 1. The mission log then has two lines.
5. Give the descriptor to lr_open, or to lr_open_follow with --follow, with own_fd 1. Give bufsize as 0 (the default) or as 256 or more: a smaller buffer refuses correct reports with EMSGSIZE. *Fail:* write the reason to standard error, close the feed and the mission log, and exit with status 1.
6. Read sc_now_ns: the elapsed time starts.
7. Read the feed to the end, and write one line to the mission log for each line of the feed (Sections 3.3 and 3.4).
8. Read sc_now_ns again: the elapsed time stops.
9. Call lr_close, then proc_report(0, &after). This is the *after* count. *Fail:* write the reason to standard error, and continue. The exit status will be 1.
10. Print the summary to standard output, then the process report with proc_report_write, then the open_fds line. If step 9 failed, print the summary only. If proc_report_write fails: write the reason to standard error, and continue. The exit status will be 1.
11. Write the last line to the mission log, and call log_close. If log_close fails: write the reason to standard error. The exit status will be 1.
12. If a line of the mission log was lost (Section 3.3), write the number of lost lines and the last reason to standard error. Then exit with the status of Section 3.5.

At steps 3 and 9, the mission log is open and the feed is closed. So the two counts are equal when the demo closed each descriptor that it opened for the feed.

3.3 The mission log

log_writef adds the timestamp and the level. The table gives the level and the exact message of each line. Text in angle brackets is a value. Write all other text as the table shows it, with one space where it shows a space. Each message is one line.

If log_writef fails, in any step, continue, and count the lost line. Write nothing at that time: on a full disk each line fails, and a message for each one hides the cause. Step 12 writes the number of lost lines and the last reason to standard error, one time. The exit status will be 1.

When	Level	Message
first line	SYSTEM	SENTINEL Part 1 starting, feed=<feed>
second line	SYSTEM	pid=<pid> ppid=<ppid> name=<name> open_fds=<fds>

When	Level	Message
a report with flag[4] `R`	`INFO`	`contact <id> from <sensor>`
a report with flag `P`	`PRIORITY`	`priority contact <id> from <sensor> at <lat>,<lon> speed <speed>`
a line that is not a report	`INFO`	`rejected line <n>: <reason>`
a read error	`SYSTEM`	`feed read failed: <error>`
last line	`SYSTEM`	`Part 1 complete: <read> read, <accepted> accepted, <rejected> rejected, <priority> priority, <ms> ms`

- `<feed>`: the feed argument, with no change. `<pid>`, `<ppid>`, `<name>`, `<fds>`: `pid`, `ppid`, `name` and `open_fds` of the *before* report.
- `<id>`, `<sensor>`, `<lat>`, `<lon>`, `<speed>`: from the record of `contact_parse`, with `%.4f` for `<lat>` and `<lon>`, and `%.1f` for `<speed>`.
- `<n>`: the number of the line in the feed. The first line is 1, and each line counts, also an empty line.
- `<reason>`: `contact_strerror` of the error of `contact_parse`. For a line that the reader refuses with `EMSGSIZE`, it is `contact_strerror(CONTACT_ERR_TOO_LONG)`, `line too long`. An empty line has the same reason.
- `<error>`: `strerror(errno)`.
- `<read>`: the number of lines in the feed. `<accepted>` plus `<rejected>` equals `<read>`. `<priority>`: the accepted reports with the flag `P`. `<ms>`: the elapsed time of steps 6 to 8 in milliseconds, with `%.3f`.

The acceptance test checks four more rules:

- From line 3, the mission log has one line for each line of the feed, in order. A feed of k lines gives $k + 3$ lines. A read error adds one line before the last line: $k + 4$.
- An empty feed gives three lines: the start line, the process line and the last line.
- A last line with no `\n` is a line for the reader of `lr_open`. The reader of `lr_open_follow` keeps it until its `\n` arrives. If the run stops first, the demo does not count it or write it.
- No line of the mission log is longer than 512 bytes, with its `\n`.

3.4 The read loop

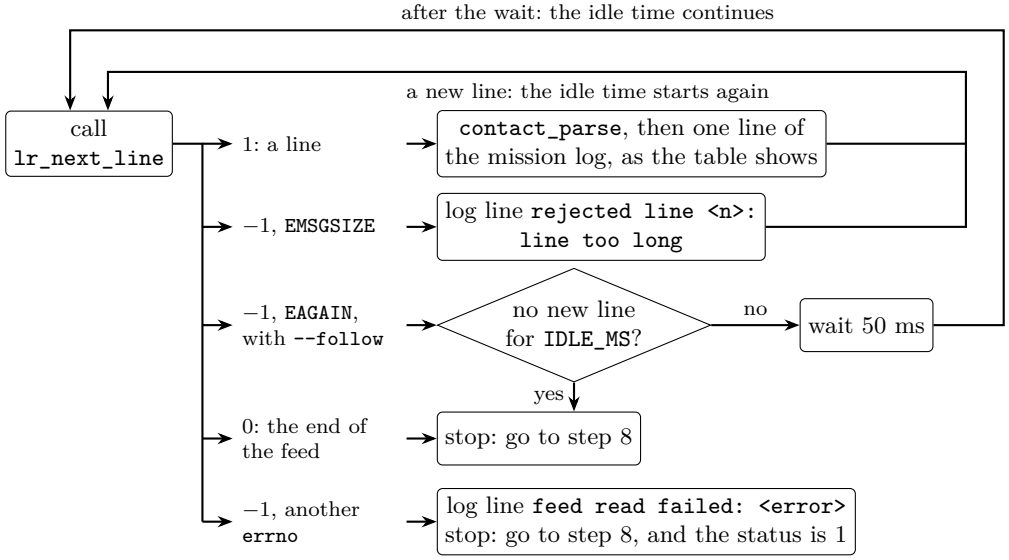


Figure 1. The read loop of step 7. Each result of `lr_next_line` has one action. `EAGAIN` means "nothing yet". `EMSGSIZE` means that one line is lost, and the next call works.

Figure 1 gives the action for each result of `lr_next_line`. These three rules of the loop give marks (Section 6.1):

[4]The flag is the last field of a contact report: `P` means urgent, and `R` means routine. The SENTINEL Lab Manual, Section 1.4, `docs/manual/sentinel_manual.pdf`, describes it.

Rule 1. With `--follow`, `EAGAIN` means that no new line is there yet. Wait 50 ms, then call `lr_next_line` again. Do not stop on `EAGAIN`.

Rule 2. With `--follow`, stop when no new line arrived for `IDLE_MS`. The idle time starts at step 6. It starts again at each new line, also at a line that the reader refuses with `EMSGSIZE`. A wait does not start it again.

Rule 3. After `EMSGSIZE`, write the line as rejected (Section 3.3), and continue. The reader drops only that line.

Each error other than `EMSGSIZE`, and other than `EAGAIN` with `--follow`, is a read error. `<read>` does not count a read error.

3.5 The standard output, and the exit status

The standard output has three required parts. The rest is your choice.

- Four summary lines, in this order: `lines read`, `accepted`, `rejected` and `priority`. Each line has its label, one or more spaces, and its number. Each number is equal to the number in the last log line. You choose the spaces at the start of the line.
- The process report, which `proc_report_write` writes from the *after* report. Call `fflush(stdout)` before it, because `printf` keeps its output in a buffer and `write` does not.
- The leak check line, with the *before* and *after* counts, and `(LEAKED)` in place of `(balanced)` when they are different. You choose the spaces at the start:
  `open_fds before=<before> after=<after> (balanced)`

The exit status is one of these:

Status	Meaning
0	The run finished, and the demo accepted at least one line, or the feed was empty
1	One of these faults: • The demo could not open the mission log or the feed. • A `proc_report` failed, or a read error stopped the loop. • The feed had lines, and the demo rejected each one. • A `log_writef`, `proc_report_write` or `log_close` failed.
2	An argument is not correct

For a read error, a failure in step 9, a failed `log_writef` or `proc_report_write`, or a feed of rejected lines, the demo still does each step. Then it exits with status 1.

4 Build, test and check

4.1 Your Makefile

1. Build `$(BUILD)/libsyscall.a` from each file in `src/`.
2. Build `$(BUILD)/demo_lab1`.
3. Give a `test` target. It builds and runs each `tests/test_*.c`, and it stops at the first failure.
4. Give a `clean` target. It removes only the files that your `Makefile` built.

Set each of these three variables with `?=`, so that your `Makefile` also works alone, after one run of `make` at the top level. The top-level `Makefile` gives the values in the middle column. Keep the line `CC ?= gcc`, because it states your default, although `make` gives `CC` the value `cc` before it reads your file. On Linux, `cc` is `gcc`.

Variable	Value from the top level	Your default
`CC`	`gcc`	`gcc`
`CFLAGS`	`-std=c11 -D_GNU_SOURCE -Wall -Wextra -Werror -g -O1`	the same
`BUILD`	the absolute path of `build/`	`$(abspath ../../build)`

Compile with `-Iinclude -I../../common`. Link each program with `$(BUILD)/libsyscall.a $(BUILD)/libcommon.a -lm`, in this order, because the linker reads from left to right. Do not remove `-Werror`. The top-level `make` builds `$(BUILD)/libcommon.a` before it runs your `Makefile`. Your `Makefile` uses it, and does not build it or remove it.

4.2 Your tests

Write tests/test_io.c, tests/test_clock.c, tests/test_log.c and tests/test_proc.c. Each is a plain C program that uses assert, and it returns 0 when each assertion passes. Do not use a test framework. A program that only prints values is not a test.

- make test runs your tests in src/libsyscall/, and the marker can run them from a different directory. So a test makes each file that it needs itself, at an absolute path under /tmp.
- Test each promise of syscall.h,[5] and each error. After each error, show that the next call still works.
- Do not test the given code in common/.

4.3 Check your work

Run make lab1[6] and make lab1 PROFILE=dirty. With dirty,[7] the summary must show a rejected count above zero, and the demo must read to the end of the feed.

No profile causes EMSGSIZE. The reader gives EMSGSIZE only for a line that is longer than its buffer. The longest line of dirty, 222 bytes, fits in a buffer of 256 bytes. To test your EMSGSIZE branch, put a line of 5000 bytes in a feed: it is longer than the default buffer of 4096 bytes. The summary must then show lines read 11, and rejected of 1 or more.

```
{ head -n 5 build/live/feed.txt
  head -c 5000 /dev/zero | tr '\0' X; echo
  tail -n 5 build/live/feed.txt
} > /tmp/long.txt
build/demo_lab1 /tmp/long.txt /tmp/long.log
```

make accept1 compiles the acceptance test, tools/accept/accept_lab1.c, with your library, and runs 34 checks, the last of them on your demo. It is a part of the tests that give the marks. Each line of its output names a source file, so you know where to look. To see exactly what a check does, read it in accept_lab1.c. Do not change that file: the marks come from the course copy of it.

5 Done when

- ☐ make builds your library and your demo with no warning.
- ☐ make test passes, and make accept1 passes all 34 checks.
- ☐ make lab1 and make lab1 PROFILE=dirty both complete, and the second shows a rejected count above zero.
- ☐ The demo prints (balanced).
- ☐ With --follow, the demo stops because no new line arrived for IDLE_MS, not because the reader gave EAGAIN.
- ☐ valgrind reports no leak and no error:

  ```
  valgrind --leak-check=full --error-exitcode=1 \
      build/demo_lab1 build/live/feed.txt /tmp/check.log
  ```

  Under valgrind, open_fds is higher, but the two counts must be equal. The process name is the name of the tool: memcheck-arm64- on aarch64 and memcheck-amd64- on x86_64. So a test compares the name with the first line of /proc/self/comm, and not with a fixed text.

[5] src/libsyscall/include/sentinel/syscall.h, in your repository. It states what each function of Lab 1 does, and each of its errors.

[6] make lab1 writes a new feed, build/live/feed.txt, from the four sensors, and it runs your demo on it. The SENTINEL Lab Manual, Section 4, docs/manual/sentinel_manual.pdf, lists each make target.

[7] A profile selects the traffic that contactfeed sends: world, ideal, dirty, field or threat. The SENTINEL Lab Manual, Section 3.1, docs/manual/sentinel_manual.pdf, describes each one.

6 Marks and the hand-in

6.1 The marks

Part of the deliverable	Marks	The rules are in
Acceptance test, `make accept1`	16	In proportion to the checks that pass
Course tests, run on your library	16	In proportion to the test programs that pass
Your own tests and edge cases	12	Section 4.2
Code style and error handling	12	Section 2, and Rules 1 to 3 of the read loop in Section 3.4
`Makefile` and `valgrind`	12	Section 4.1, and the `valgrind` item of Section 5
Agreement of the three files	12	Section 6.2
Total	**80**	

You do not get the course tests. They check only the rules of `syscall.h`[8] and of this brief.

The Submission Guide[9] states the form of the oral exam. In Lab 1, it has four questions, for 20 marks. Two questions are about the theory. One asks you to predict what your code does. One asks what a change to one line of your code does, and why. The examiner can ask each question of the prelab, `docs/lab1/prelab1.pdf`.

6.2 The three files of one run

Put `demo_output.txt` (the standard output), `missions.log` (the mission log) and `proc_capture.txt` (a capture of `/proc` while the demo runs) in `docs/lab1/`. All three come from **one** run. Run both commands in `bash`, each in a terminal. The run takes about 20 seconds. The first command runs `contactfeed`[10] with the profile `world`.[11]

```
rm -f docs/lab1/missions.log
build/demo_lab1 <(build/contactfeed --sensor NWS --rate 20 --count 400 \
    --profile world) docs/lab1/missions.log > docs/lab1/demo_output.txt
```

While it runs, in a second terminal:

```
pid=$(pgrep -x demo_lab1)
{ grep -E '^(Name|Pid|PPid|Threads|VmRSS)' /proc/$pid/status
  echo
  ls -l /proc/$pid/fd
} > docs/lab1/proc_capture.txt
```

With `<(...)`, the feed is a pipe, and three results are correct. The start line names a feed such as `/dev/fd/63`. The process line shows `open_fds=5`, because `bash` opened descriptor 63 for the demo. The capture shows descriptors 0 to 4 and 63. Descriptors 0 and 2 are the terminal only when you start the demo in a terminal. If not, they show where its standard input and standard error go.

The marker checks that the three files agree. Check each point before you continue:

- `Name:` in the capture is `demo_lab1`.
- The pid in the capture is the pid on line 2 of the mission log.
- The four numbers of the summary in `demo_output.txt` are the numbers of the last line of the mission log.
- The leak check line says `(balanced)`, and its `before` number is `open_fds` on line 2 of the mission log.
- The capture has one descriptor more than `open_fds` on line 2. The extra one is the feed: the demo writes line 2 at step 3, and opens the feed at step 4.

Then run `make submit`: the Submission Guide gives the steps.

[8] `src/libsyscall/include/sentinel/syscall.h`, in your repository. It states what each function of Lab 1 does, and each of its errors.

[9] The Submission Guide, `docs/submit/submit.pdf` in your repository. It gives the marks of each lab, the oral exam, and the steps of the hand-in.

[10] `contactfeed` is the given program that writes contact reports in place of the four sensors. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes it.

[11] A profile selects the traffic that `contactfeed` sends: `world`, `ideal`, `dirty`, `field` or `threat`. The SENTINEL Lab Manual, Section 3.1, `docs/manual/sentinel_manual.pdf`, describes each one.